

The Automated Training Pipeline

Part 2: take the generated dataset and train an adapter through a predefined, configurable pipeline that preserves the model's quality

Recipient: Ian

Date: June 2026

Follows: the dataset-generation build task (Part 1)

Mode: research plus experimentation. Not final production implementation.

1. How this fits

Part 1 had you build the half that turns a linked knowledge base plus a target task into a task-specific training dataset. This is the other half: an automated, predefined pipeline that takes that dataset and trains an adapter, with light steering and with the model's general quality protected throughout. Together the two halves are the whole thing, a system that goes from a body of knowledge and a target task to a trained, validated adapter, automatically.

This part is research plus experimentation, not final implementation. The aim is that you understand how teams build automated, reproducible training pipelines and how they keep quality from slipping, that you've looked at real reference implementations, and that you've wired up a small working version on top of your Part 1 output. Don't polish a production system, build the experimental loop and form a clear view of how it should work.

2. The vision

Generated dataset in, predefined automated training pipeline, trained adapter out. The pipeline has a configuration control point that works like the data-selection proposal in Part 1: an agent or a set of rules proposes the training configuration, the user approves or adjusts it, and then the run proceeds. Quality preservation is built into the run, and a two-axis evaluation acts as the gate that accepts or rejects the result. The point is that this is repeatable and largely hands-off, the same pipeline runs for any task whose dataset Part 1 produced.

3. The pipeline stages

3.1 Configure (agent or rules propose, user confirms)

Given the task and the generated dataset, the system proposes a training configuration: base model, LoRA or QLoRA settings (rank, alpha, target modules), the data mix or replay ratio, epochs, learning rate, sequence length. The user approves or adjusts, exactly the steer-then-scale pattern from Part 1. Sensible rule-based defaults are fine as the starting point; an agent proposing them is the layer on top. The key is that configuration is a defined, reviewable step, not buried in a script.

3.2 Train (config-driven)

Run the configured LoRA or QLoRA training through a pipeline that is driven by that configuration rather than ad hoc code. This is what makes runs reproducible and the whole thing automatable: the config is the single source of truth for the run.

3.3 Quality guardrails (built into the run)

Preserving the model's general ability is not a post-check, it is built into how you train. The established levers, which you've met before, belong here as pipeline settings: a replay mix of general and reasoning and tool-calling data so those abilities survive, a smaller learning rate, regularization toward the base, early stopping driven by a general-capability metric rather than only task loss, and optionally merging the adapter's effect back toward the base. The pipeline should treat these as first-class configuration, not afterthoughts.

3.4 Evaluate (the acceptance gate)

Two axes, as the gate. Task axis: the held-out gold set, base versus fine-tuned. Regression axis: standard benchmarks plus a reasoning check and a tool-calling check, before and after on identical sets. Acceptance criteria are set in advance, a minimum task gain and a maximum tolerated regression. The adapter is accepted only if it clears both; otherwise the pipeline adjusts (mix, learning rate, epochs) and re-runs. This gate is what makes the automation trustworthy.

3.5 Output

The accepted adapter, with its config and its evaluation numbers recorded, ready to be served. Because the run was config-driven and gated, you can reproduce it and trust it.

4. Reference implementations to study

Look closely at how others build configuration-driven, reproducible training pipelines. This is the strongest signal for how a predefined pipeline should be structured, so spend real time in these.

Axolotl — the canonical config-driven pipeline. A single YAML file drives the whole chain: dataset preprocessing, training, evaluation, quantization, and inference, and runs are reproducible by sharing the config. Supports LoRA/QLoRA, reasoning (GRPO), and reward methods. This is the primary reference for what you're building:

github.com/axolotl-ai-cloud/axolotl.

LLaMA-Factory — a unified framework over 100+ models with a zero-code CLI and a web UI, config-driven, covering LoRA/QLoRA, full fine-tuning, and reward methods:

github.com/hiyouga/LLaMA-Factory.

torch tune — PyTorch-native YAML recipes for full/LoRA/QLoRA training, for when you want full control without opaque abstractions: github.com/pytorch/torch tune.

TRL — the underlying trainer library (SFT, DPO, GRPO) that several of the above build on: huggingface.co/docs/trl. And Unsloth, which you're already using, for fast single-GPU runs on Kaggle/Colab: github.com/unslothai/unsloth.

Quality-preserving pipeline patterns

Beyond the frameworks, study how real domain-adaptation projects keep quality. A common, effective pattern is a staged curriculum: train on general or replay data first, then on the domain or task data, so the model gains the task without forgetting. Look for published reference projects that fine-tuned a domain specialist and reported holding their general benchmarks, and note exactly what mix, learning rate, and stopping rule they used. Use the deep-research mode of an AI assistant to map what's market-standard here and where the limits are.

5. What to research and experiment with

1. **Map the landscape.** How are automated, reproducible training pipelines built (config-driven), and how is quality preserved (replay ratios, merging, early stopping on a general metric)? Prepare pointed research questions and push them through deep-research mode to find approaches, options, and limitations.
2. **Wire a small config-driven run.** On top of your Part 1 generated dataset, run a small LoRA training driven by a configuration (via Axolotl, or via your Unsloth setup on Kaggle), so the run is defined by config rather than ad hoc code.
3. **Exercise the guardrails.** Train once without the replay mix and once with it, and compare the regression axis, so you see the quality-preservation effect with your own numbers.
4. **Run the gate.** Measure both axes against pre-set acceptance criteria, and let a failed gate trigger one adjusted re-run. That loop is the heart of the pipeline.

6. What to produce

A small working version of the train-then-evaluate loop driven by a configuration, with the quality guardrails in place and the two-axis gate deciding accept or reject. Plus a short write-up: the reference implementations you studied and what you took from them, how you'd structure the configuration step (rules now, agent later), which quality-preservation levers you used and what they did to the regression numbers, and the base-versus-fine-tuned results for one task with the gate's verdict.

Then a short sketch of the whole module end to end, now that you've built both halves: knowledge base and target task in, dataset generated, adapter trained and gated, ready to serve, and where the human or agent steers along the way. Keep it research and experimental, not a final build. Run small on Kaggle and Colab, and lean on Claude throughout to study, wire things up, and pressure-test your approach.